



TECHNICAL REPORT · EMBEDDED SYSTEMS & ROBOTICS

Autonomous RC Car: Monocular SLAM Navigation

École Supérieure en Informatique — Sidi Bel Abbès
2CS — End of Year Project

PRESENTED BY

Lakhdari Anis Charef Eddine

SUPERVISED BY

Abdelatif RAHMOUN

Hamdane BENSENANE

Academic Year 2025 / 2026

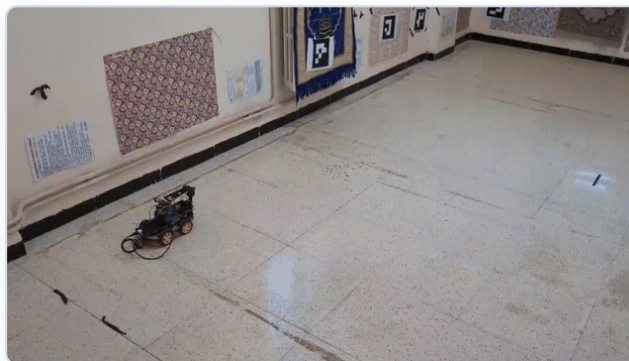
Project Repository & Demonstrations

GITHUB REPOSITORY

github.com/LakhdariAnis/monocular-slam-navigation

The complete source code, hardware specifications, and custom modifications to the ORB-SLAM3 framework are available in the repository. Below are visual demonstrations of the autonomous system in operation.

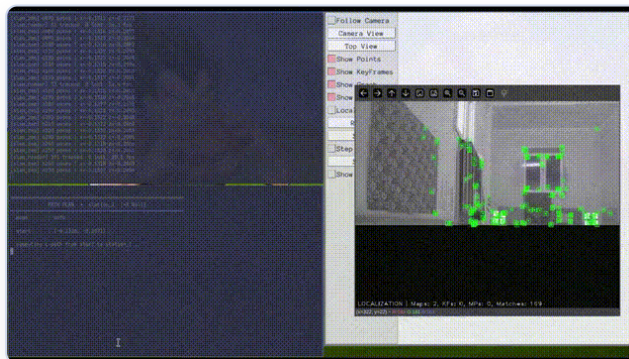
Autonomous Navigation



Autonomous navigation in action, demonstrating the L-path planning approach and ArUco-based absolute relocalization within the mapped environment.

Click image to view animation.

Autonomous Navigation in Action 2



Screen record of real-time navigation.

Click image to view animation.

TABLE OF CONTENTS

—

Abstract

1

Project Overview & Goals

1.1

Objectives

1.2

Navigation Strategy

1.3

Hard Constraints

2

System Architecture

2.1

Two-Device Pipeline

2.2

Component Descriptions

2.3

ZMQ Socket Map & Pose Format

3

Hardware Setup

3.1

Bill of Materials

3.2

Power Architecture

3.3

Motor & Drive System

3.4

GPIO & Wiring Map

3.5

Why SLAM Is Offloaded

3.6

ArUco Marker Setup

4

SLAM & Navigation Details

4.1

Monocular ORB-SLAM3

4.2

ArUco Relocalization State Machine

4.3

L-Path Planning

4.4

IMU Integration & Heading

4.5

Spin Protocol — "Shoot & Recover"

4.6

Navigation Parameters

4.7

Map Builder Pipeline

5

Key Engineering Challenges & Solutions

5.1

Motion Blur & Spin Speed Control

5.2

Post-Spin SLAM Settlement

5.3

Silent Motor Stall Detection

5.4

SLAM Loss in Feature-Poor Rooms

5.5

IMU Heading Drift

6

ORB-SLAM3 Custom Modifications

6.1

New Features Added

6.2

Bug Fix 1 — Atlas Serialization Crash

6.3

Bug Fix 2 — Cross-Map Relocalization

6.4

Bug Fix 3 — Shutdown Race Condition

7

Results & Demo

—

References

ABSTRACT

This report presents the design and full implementation of an autonomous RC car that navigates between calibrated stations in an indoor room using monocular visual-inertial SLAM. The system combines a custom build of ORB-SLAM3 — extended with ArUco-based absolute relocalization and an external pose injection API — with an MPU-6050 IMU, L-path planning, and a ZMQ pub/sub data pipeline running over WiFi. Because ORB-SLAM3 requires approximately 800 MB at runtime, SLAM compute is offloaded to a PC; the Raspberry Pi 3B (1 GB RAM) handles only sensor streaming and motor control.

Hardware comprises four TT DC motors driven by two L298N dual H-bridge modules, a 12 V battery pack for the motor rail, a 5 V powerbank for Pi and logic, an MPU-6050 six-axis MEMS IMU, and a Pi Camera v2. The report covers system architecture, five key engineering challenges with their solutions, and three critical ORB-SLAM3 bug fixes required for reliable atlas persistence and cross-session localization. A software-only demo mode requires no hardware.

SECTION 01

Project Overview & Goals

1.1 Objectives

The project achieves station-to-station autonomous navigation in an indoor room using the minimum possible sensor suite: a single monocular camera (Pi Camera v2) and a 6-axis IMU (MPU-6050). There is no lidar, no wheel encoder, no depth sensor, and no external positioning system such as GPS or motion capture.

The car navigates between calibrated stations — fixed points around the room, each associated with a wall-mounted ArUco marker — using an **L-path planner** that decomposes every move into a forward segment, a 90° spin, and a second forward segment. This perpendicular-approach strategy ensures the car always meets the target wall head-on, simplifying stall detection and SLAM pose validation.

1.2 Navigation Strategy

The L-path planner decomposes every station-to-station move into two axis-aligned legs connected by a single spin. The car travels in one SLAM coordinate axis at a time, making residual-distance checks straightforward and stall detection reliable.

A web dashboard served at <http://localhost:8080> animates the full L-path, tracks SLAM position in real time, and updates the navigation phase indicator. Two modes are supported: **real mode** (live Pi hardware and ZMQ sensors) and **test mode** (no hardware required — mock sensors and simulated motion).

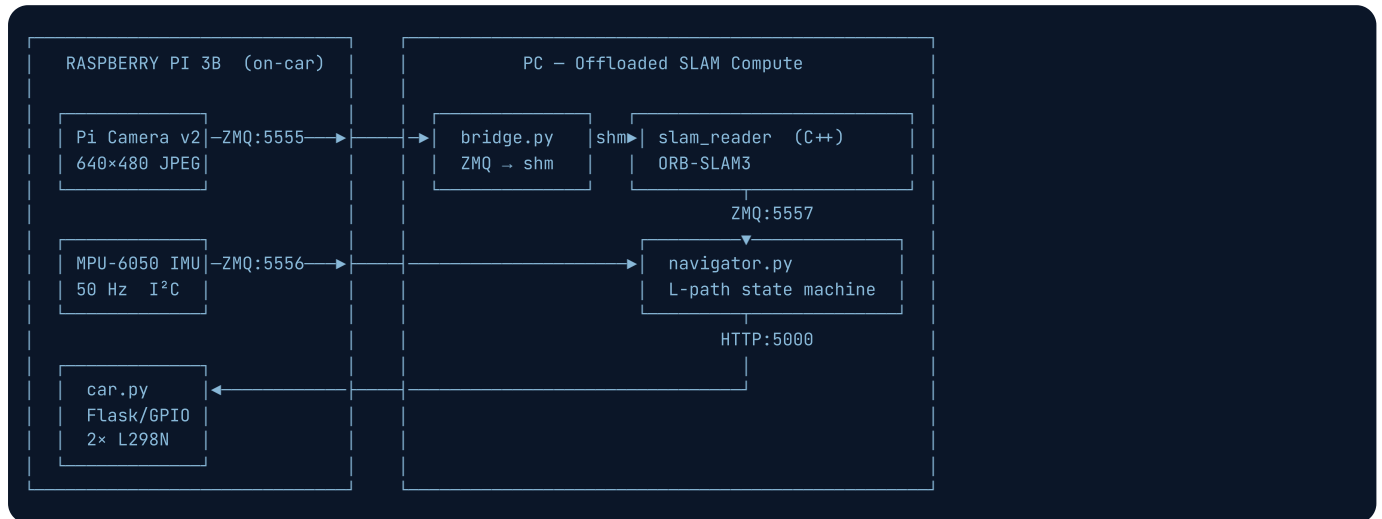
1.3 Hard Constraints

- **Hardware constraint:** A Raspberry Pi 3B at 1 GB RAM cannot run ORB-SLAM3 in real time (~800 MB at runtime leaves no headroom for the OS or sensor loops). All SLAM compute is offloaded to a connected PC over WiFi.
- **Sensor constraint:** Without encoders, the only source of heading is the IMU. Without depth, the only source of position is SLAM. Both must remain stable through 90° spin maneuvers at each waypoint.

System Architecture

2.1 Two-Device Pipeline

The system is split across two devices — the Raspberry Pi (on the car) and a PC running SLAM — connected over local WiFi. Sensor data flows from the Pi to the PC over ZMQ pub/sub sockets; motor commands return over HTTP. POSIX shared memory decouples network I/O from the C++ SLAM thread on the PC side, preventing a slow wireless link from stalling the tracker.



2.2 Component Descriptions

BRIDGE.PY

Subscribes to the Pi's ZMQ PUB on port 5555, decodes JPEG frames to BGR via OpenCV, and writes each frame plus a sequence number and timestamp into POSIX shared memory (`/orbframe` , 921,616 bytes). Decoupling network I/O from the C++ SLAM thread prevents a slow WiFi link from stalling the tracker.

SLAM_READER (C++)

Main SLAM binary. Reads frames from shared memory using a double-read consistency check on the sequence number, optionally detects ArUco markers and stamps their corners as ORB features, feeds each frame to `ORB_SLAM3::System::TrackMonocular` , and publishes the SE3 pose as JSON on ZMQ port 5557 at up to ~30 fps.

SLAM_ZMQ.PY

Python supervisor: launches `slam_reader` as a subprocess, parses its stdout for tracked-pose and tracking-lost events, and relays enriched JSON (with a `reset` flag) on port 5557. Maintains a REP socket on port 5558 for the navigator ACK handshake after map resets.

NAVIGATOR.PY

Subscribes to SLAM poses (port 5557) and IMU heading (port 5556). Executes the L-path state machine and issues motor commands over HTTP to `car.py` . Contains a `ColorDetector` module for fine-grained station alignment using visual color cues.

CAR.PY

Flask HTTP server on the Pi on **port 5000**. Receives JSON drive commands and actuates the L298N H-bridge drivers via `RPi.GPIO` PWM at 1 kHz. Format: `{"w":true,"a":false,"s":false,"d":false,"inner":30}` where `w/a/s/d` map to forward/spin-left/reverse/spin-right.

2.3 ZMQ Socket Map

SOCKET	DIRECTION	PUBLISHER	SUBSCRIBER	CONTENT
tcp://PI_IP:5555	Pi → PC	camera_zmq.py	bridge.py	Raw JPEG bytes (640×480)
tcp://PI_IP:5556	Pi → PC	imu_zmq.py	navigator.py	JSON: heading_deg , moving
tcp://localhost:5557	PC → PC	slam_reader	navigator.py	JSON: x,y,z,R[9],ok,seq,ts
http://PI_IP:5000/drive	PC → Pi	navigator.py	car.py	JSON: {w,a,s,d,total,inner}

i ZMQ_CONFLATE=1 on all subscribers
All ZMQ subscribers use `CONFLATE=1` — only the latest message per socket is retained, preventing queue backlog under SLAM latency spikes.

```
JSON • ZMQ port 5557 • ~30 fps

// Tracking OK
{"seq":1234,"ts":1720000000.123,"ok":true,
 "x":-0.045,"y":-0.081,"z":0.430,
 "R":[r00,r01,r02,r10,r11,r12,r20,r21,r22]}

// Tracking lost
{"seq":1235,"ts":1720000000.157,"ok":false,"x":null,"y":null,"z":null,"R":null}

// Map reset (injected by slam_zmq.py after atlas reload)
{"seq":-1,"ts":0.0,"ok":false,"x":null,"y":null,"z":null,"reset":true}
```

Hardware Setup

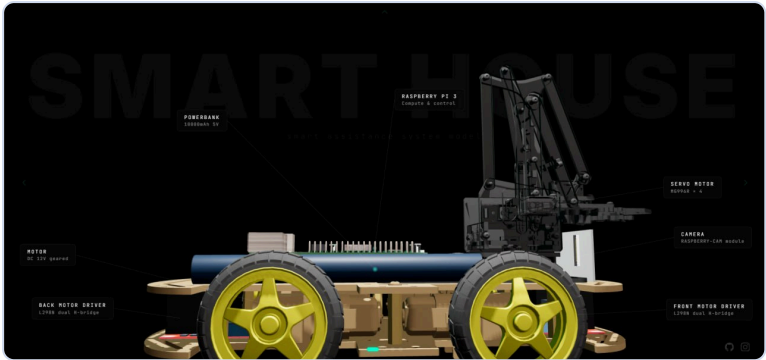


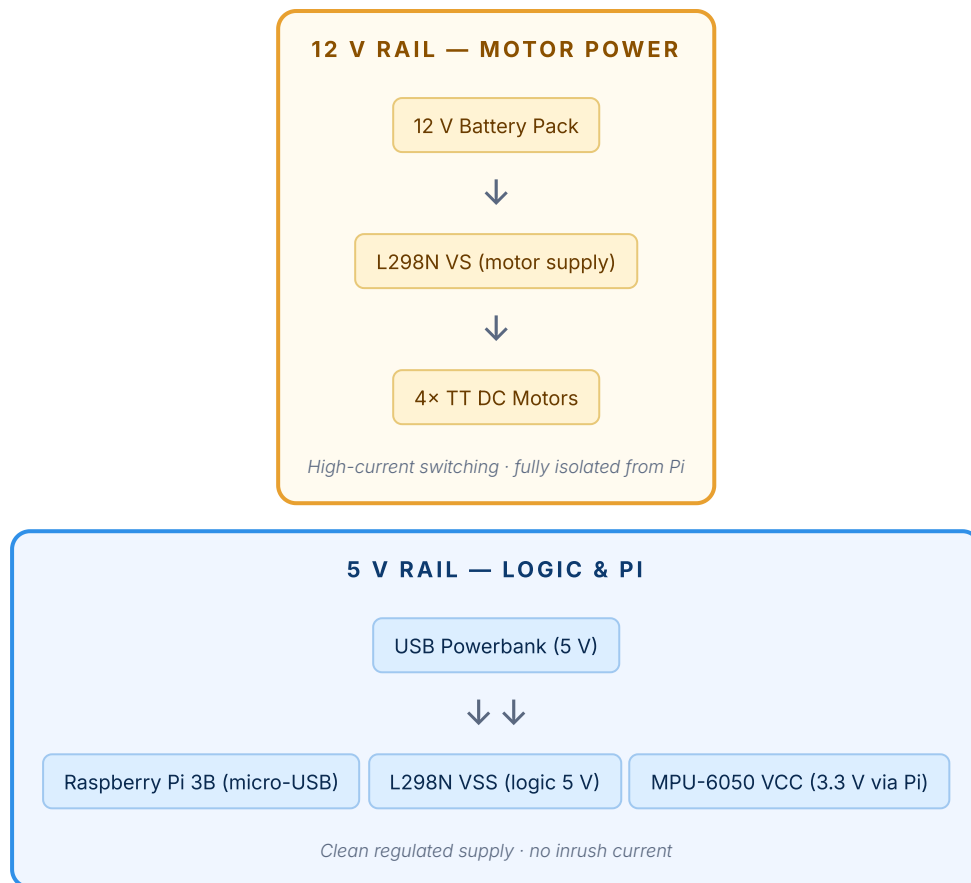
Figure 1. Rendered view of the autonomous RC car platform — Pi Camera v2 (front-facing), MPU-6050 IMU (top deck), dual L298N H-bridge drivers, four TT DC motors in skid-steer layout.

3.1 Bill of Materials

COMPONENT	PART / SPECIFICATION	ROLE IN SYSTEM
Drive Motors	4× TT DC motors (yellow gearbox, ~1:48 reduction)	4-wheel skid-steer differential drive
Motor Drivers	2× L298N dual H-bridge modules	Bidirectional PWM motor control at 1 kHz (up to 2 A/channel)
Motor Power	12 V battery pack	High-current rail for all 4 TT motors via L298N VS
Logic / Pi Power	5 V USB powerbank	Pi 3B micro-USB input and L298N VSS 5 V logic rail
On-board Compute	Raspberry Pi 3B — 1 GB RAM, quad-core A53 @1.2 GHz	Sensor streaming and motor HTTP server only (no SLAM)
Vision	Pi Camera v2 (Sony IMX219, 8 MP)	640×480 capture, JPEG quality 80, streamed on ZMQ:5555
IMU	MPU-6050 (6-axis MEMS — 3-axis gyro + 3-axis accel)	I ² C bus 1, addr 0x68 , 50 Hz heading via complementary filter
Localization Assist	ArUco markers — DICT_4X4_50, printed at 18.4 cm	Printed, laminated, wall-mounted absolute pose references (IDs 0–7)
SLAM Compute	PC / Laptop (any modern x86-64 CPU)	ORB-SLAM3 C++ + Python navigator — offloaded from Pi

3.2 Power Architecture

Two isolated power rails prevent motor switching noise from interfering with the Pi’s sensitive 3.3 V logic and I²C bus:



⚠ Shared ground is mandatory

The 12 V battery GND and the powerbank GND must be tied together at the L298N board. Without a common ground reference, the PWM signal levels seen by the H-bridge are undefined and motor control fails unpredictably.

3.3 Motor & Drive System

Four TT DC motors are arranged in a skid-steer (tank-drive) configuration. Two L298N dual H-bridge modules each control one pair of motors:

- **Front L298N:** Front-Right (FR) and Front-Left (FL).
- **Rear L298N:** Rear-Left (RL) and Rear-Right (RR).

Both motors on the same side always receive identical PWM. On-axis spins (`a` or `d`) counter-rotate left and right pairs at equal duty cycles. The `inner` parameter adjusts inside-wheel speed for arc turns (default 30%).

⚠ Motor dead zone — critical design constraint

TT motors exhibit a stall threshold at approximately **40% PWM duty cycle** on carpeted surfaces. Below this value, static friction exceeds motor torque and the car stalls silently with no error signal. All motion commands are clamped to a minimum of 40% PWM.

3.4 GPIO & Wiring Map

MOTOR	ENABLE (PWM, 1 KHZ)	FORWARD IN	REVERSE IN
Front-Right (FR)	GPIO 12 (ENA)	GPIO 17 (IN1)	GPIO 27 (IN2)
Front-Left (FL)	GPIO 13 (ENB)	GPIO 22 (IN3)	GPIO 23 (IN4)
Rear-Left (RL)	GPIO 18 (ENA2)	GPIO 24 (IN5)	GPIO 25 (IN6)
Rear-Right (RR)	GPIO 19 (ENB2)	GPIO 5 (IN7)	GPIO 6 (IN8)

MPU-6050: SDA → GPIO 2 · SCL → GPIO 3 · VCC → 3.3 V · AD0 tied low → I²C address `0x68`.

Pi Camera v2: 15-pin CSI flat cable into the Pi's CSI-2 port. Capture format set to `BGR888` — no channel-swap needed before handing frames to ORB-SLAM3.

3.5 Why SLAM Is Offloaded

ORB-SLAM3 requires approximately **800 MB of RAM** at runtime. The Raspberry Pi 3B has only 1 GB total; after OS overhead and the three sensor-streaming services, essentially no margin remains. Running SLAM on-board would trigger OOM kills or swap-thrashing, collapsing real-time pose estimation.

3.6 ArUco Marker Setup

Eight markers (IDs 0–7, DICT_4X4_50, printed at 18.4 cm) are laminated and taped to the room's four walls — two per wall. Their 3D poses in SLAM coordinates are computed during a dedicated mapping pass and saved to `MarkerPositions.txt`. In localization mode, each visible marker provides an absolute camera pose in under one frame via PnP, enabling instant relocalization with no BoW vocabulary matching.

▲ Corner-stamping radius must match across sessions

White squares of radius `ARUCO_CORNER_RADIUS = 6 px` are painted over every marker corner before ORB extraction. This radius must be *identical* in both mapping and localization sessions — a mismatch produces different descriptors and breaks BoW map matching.

SLAM & Navigation Details

4.1 Monocular ORB-SLAM3

The system uses monocular visual SLAM — a single camera, no depth sensor. ORB-SLAM3 extracts ORB features from each frame, tracks them across frames to estimate camera motion, and maintains a sparse 3D map via local bundle adjustment and loop closure. Because monocular SLAM is scale-ambiguous, the reconstructed map is calibrated to room coordinates via a post-processing 2D affine transform anchored to known physical marker distances, computed by `map_builder.py`.

Two operating modes:

- **Mapping (`--map`):** Builds a fresh map from the live feed. On Ctrl+C, saves `room_map.osa`, `MapPoints.txt`, `MarkerPositions.txt`, and `KeyFrameTrajectory.txt`, then auto-invokes `map_builder.py`.
- **Localization (`--loc <atlas>`):** Loads a saved atlas, activates `SLAM.ActivateLocalizationMode()`, streams live poses on ZMQ port 5557, and recovers via ArUco without modifying the map on disk.

4.2 ArUco Relocalization State Machine

Standard BoW-based relocalization fails in plain-painted-wall rooms. ArUco markers provide absolute camera pose in a single frame:

1. `cv::aruco::detectMarkers` scans each incoming frame for DICT_4X4_50 markers.
2. On the first lost frame where a known marker is visible, `SLAM.DeactivateLocalizationMode()` is called — allowing ORB-SLAM3 to add keyframes and rebuild local map context.
3. `cv::aruco::estimatePoseSingleMarkers` runs PnP on the four marker corners to obtain `T_cm`. Combined with stored `T_wm`:

```
T_wc = T_wm × T_cm-1      (camera in world coords)
Tcw  = T_wc-1           (ORB-SLAM3 convention: world-in-camera)
```

4. `SLAM.InjectPoseHint(Tcw)` writes `Tcw` into the tracker's motion model, zeros velocity, and sets state to `Tracking::OK`.
5. `ActivateLocalizationMode()` re-engaged once `GetTrackingState()` returns 2.
6. If recovery fails within `RELOC_MAX_FRAMES = 60` (~2 s), the state machine resets and retries on the next lost frame with a visible marker.

4.3 L-Path Planning

Every station-to-station move is decomposed into two axis-aligned legs connected by a single 90° spin. Wall classification determines leg order:

- **±X-wall stations:** walk Z-axis first, then X.
- **±Z-wall stations:** walk X-axis first, then Z.

If the lateral offset is below `STRAIGHT_THRESHOLD = 0.03 m`, the planner skips the elbow and drives straight. The navigator switches to slow speed within `SLOW_DISTANCE = 0.20 m` and stops within

`WAYPOINT_THRESHOLD = 0.02 m` of the elbow or `STATION_THRESHOLD = 0.015 m` of the final standoff.

4.4 IMU Integration & Heading

The MPU-6050 provides heading via a complementary filter running at 50 Hz:

```
Python - imu_zmq.py · 50 Hz complementary filter

ALPHA = 0.98    # gyro: trusted for 98% (short-term accuracy)
                # accel: 2% (arrests long-term drift)

heading = ALPHA * (heading + gyro_z * dt) + (1 - ALPHA) * accel_heading
heading = heading % 360
```

On startup, 200 samples at rest are averaged to compute per-axis bias. A **ZUPT** (Zero-velocity UPdaTe) implemented via a sliding-window variance detector on `ax/ay` freezes integration while the car is stationary — eliminating drift during SLAM-processing pauses.

i IMU primary during spins; SLAM primary while driving

During 90° rotations, SLAM position is discarded due to motion blur. The navigator tracks heading exclusively via IMU delta and stops when ~84° is reached (6° coasting leeway). Between spins, SLAM position is primary; IMU supplies only the `moving` flag.

4.5 Spin Protocol — "Shoot & Recover"

1. **Record pre-spin position** — last known valid SLAM `(x, z)`.
2. **Spin blindly via IMU** — constant `SPIN_SPEED = 40% PWM`, all SLAM updates discarded, stop when IMU reads ~84°.
3. **Two mandatory holds:** `POST_SPIN_HOLD_SECS = 1.0 s` (inertia / rollback) then `POST_SPIN_SLAM_WAIT = 0.5 s` (ORB feature reacquisition).
4. **Drift sanity check** — compare post-spin SLAM vs pre-spin position. Drift ≤ 0.15 m: accepted. Drift > 0.15 m: rejected.
5. **Fallback** — if sanity check fails: reverse ~0.4 s at 40% PWM, wait 1.5 s, revalidate.

4.6 Navigation Parameters

PARAMETER	VALUE	DESCRIPTION
<code>SPIN_SPEED</code>	40%	PWM duty cycle for all 90° spins
<code>FORWARD_SPEED</code>	45%	Normal forward drive PWM
<code>SLOW_SPEED</code>	40%	Near-waypoint approach PWM
<code>SPIN_TOLERANCE</code>	5°	Spin complete when within N° of target
<code>WAYPOINT_THRESHOLD</code>	0.02 m	Elbow arrival distance
<code>STATION_THRESHOLD</code>	0.015 m	Final standoff arrival distance
<code>STANDOFF_DIST</code>	0.30 m	Inset from wall station marker
<code>POST_SPIN_HOLD_SECS</code>	1.0 s	Motor-stop rollback prevention
<code>POST_SPIN_SLAM_WAIT</code>	0.5 s	SLAM feature reacquisition window
<code>STALL_TIMEOUT</code>	3.0 s	Seconds before stall recovery triggers
<code>SLOW_DISTANCE</code>	0.20 m	Switch to SLOW_SPEED within this range
<code>STRAIGHT_THRESHOLD</code>	0.03 m	Lateral offset below which path goes straight

4.7 Map Builder Pipeline

`map_builder.py` is auto-invoked after every mapping session:

1. Loads `MarkerPositions.txt`; filters markers with ≥ 50 observations.
2. Solves a **2D affine transform** (anisotropic — monocular SLAM X and Z spans differ) from SLAM coordinates to room centimetres using markers 0, 4, 6, 7 as tape-measure ground-truth anchors.
3. Snaps each marker to its assigned wall and computes a 20 cm inset approach point.
4. Filters the SLAM point cloud and rasterizes into a `.pgm` occupancy costmap.
5. Writes: `data/stations.json`, `data/map_final.png`, `data/map.pgm`, `data/map.yaml`.

Key Engineering Challenges & Solutions

Five problems consumed the majority of development time. Each required a solution fundamentally different from the first attempt. Rejected approaches are documented alongside the working solutions.

5.1 Motion Blur & SLAM Tracking Loss During Spins

× Problem

At 60%+ PWM, the Pi Cam v2 produces motion blur on spinning frames. ORB feature extraction fails on blurred frames and ORB-SLAM3 loses tracking, teleporting the estimated pose by 0.5 m or more in the map — making the post-spin position completely unreliable.

✓ Solution — constant low-speed spin + IMU-only heading

All spins run at exactly `SPIN_SPEED = 40% PWM`. SLAM is completely ignored during the spin — heading tracked exclusively via IMU delta, stopping at $\sim 84^\circ$ (6° coasting leeway).

Rejected: Pulsing/coasting — heavy jitter. Dynamic ease-in/out with IMU angular-rate feedback — oscillated between stalling and over-spinning. Ultra-slow speed ($<30\%$ PWM) — stalled on carpet.

5.2 Post-Spin SLAM Settlement

× Problem

After spinning, ORB-SLAM3 needs stationary frames to reacquire features and stabilize its pose estimate. Driving immediately on a stale or mid-relocalization pose caused the navigator to target a wrong position.

✓ Solution — two unconditional sequential settlement holds

`POST_SPIN_HOLD_SECS = 1.0 s` — prevents motor-stop mechanical rollback from corrupting the post-spin pose.

`POST_SPIN_SLAM_WAIT = 0.5 s` — feature reacquisition window. Only after both holds complete does the navigator re-read SLAM and continue.

5.3 Silent Motor Stall on Carpet

× Problem

TT DC motors stall silently on carpet below ~40% PWM. The car appears to run — motor command issued, no error returned, GPIO toggling — but makes zero physical progress. Without encoders there is no direct way to detect this condition.

✓ Solution — SLAM-based sliding-window stall detector

A sliding window monitors SLAM position change per control loop step. If progress is `< 2 mm per step` for `80 consecutive steps` (~8 s), the navigator triggers a reverse-and-retry: reverses at 40% PWM for ~0.4 s, waits for SLAM to settle, then resumes.

5.4 SLAM Loss in Feature-Poor Rooms

× Problem

In rooms with plain painted walls, ORB-SLAM3's BoW-based relocalization takes several seconds or fails entirely — the vocabulary finds no repeatable visual matches. During this window the car has no position estimate.

✓ Solution — ArUco absolute relocalization via InjectPoseHint API

Wall-mounted ArUco markers provide absolute camera pose in under one frame via PnP. A custom `InjectPoseHint(Tcw)` C++ API added to ORB-SLAM3 writes the ArUco-derived pose directly into the tracker's motion model. Relocalization latency: < 1 frame (~33 ms at 30 fps).

× **Problem**

Gyroscope integration accumulates sensor noise as heading drift. During stationary SLAM-processing pauses, the gyro continues integrating noise unchecked, producing spurious heading changes of several degrees per minute.

✓ **Solution — complementary filter ($\alpha = 0.98$) + ZUPT**

The complementary filter bounds heading drift to acceptable levels for spin durations under ~5 s. ZUPT via accelerometer variance detection freezes gyro integration when the car is stationary, eliminating drift accumulation during all SLAM-processing pauses.

The current complementary filter is encoder-less and accumulates heading drift over long spin sequences (>5 s), which is acceptable for the current room-scale navigation but would degrade over longer corridors or multi-room maps. Three concrete mitigations are identified for future work, in order of implementation effort:

- **ZUPT-gated drift reset:** already partially implemented (ZUPT suppresses drift during stationary phases); extend it to also reset the heading baseline after each confirmed dock, using the known station heading stored in MarkerPositions.txt as ground truth.
- **ArUco heading correction:** after every successful ArUco relocalization, the recovered SE3 pose contains a ground-truth heading — use it to correct the complementary filter's accumulated heading error directly, closing the loop between SLAM and IMU.
- **Longer term — wheel odometry or encoder integration:** if the RC car platform is upgraded with encoders, a full EKF fusing IMU + odometry + SLAM would eliminate the drift structurally. This is tracked as a hardware upgrade path, not a software-only fix.

ORB-SLAM3 Custom Modifications

Upstream ORB-SLAM3 was unsuitable in two fundamental ways: no external pose injection API (making fast ArUco-based relocalization impossible) and atlas serialization crashed on every map-saving attempt (making cross-session localization impossible). Three bug fixes and five new features were implemented in the C++ codebase.

6.1 New Features Added

ArUco Marker Relocalization

Each incoming frame is scanned with `cv::aruco::detectMarkers`. White squares of radius `ARUCO_CORNER_RADIUS = 6 px` are stamped over detected corners so they appear as strong repeatable ORB features. In localization mode when tracking is lost and a known marker is visible: (1) switch to full SLAM mode, (2) compute pose via PnP, (3) inject via `InjectPoseHint`, (4) re-engage localization mode once tracking returns to OK.

External Pose Injection API

Signature: `bool System::InjectPoseHint(const Sophus::SE3f& Tcw);`

Writes `Tcw` into `mpTracker->mLastFrame.pose`, zeros the velocity model (`mVelocity`), and sets tracking state to `Tracking::OK`. The next `TrackMonocular` call enters `TrackWithMotionModel` from the injected position rather than a stale extrapolation.

Dual Operating Modes, ZMQ Publisher & MarkerPositions Output

`--map` / `--loc <atlas>` flags cover the full two-session workflow. JSON SE3 poses are published on ZMQ port 5557 at ~30 fps with fields `seq, ts, ok, x, y, z, R[9]`. Per-frame ArUco SE3 observations are averaged and saved to `MarkerPositions.txt` during mapping.

6.2 Bug Fix 1 — Atlas Serialization Crash

Symptom: `SLAM.Shutdown()` with `SaveAtlasToFile` set caused a segfault inside `Map::PreSave`, crashing before the atlas was written — making every `--map` session fail.

Root cause: `Map::PreSave` iterated directly over `mmpMapPoints` while calling `pMPi->EraseObservation(pKF)` inside the same loop. `EraseObservation` calls `SetBadFlag → EraseMapPoint`, which erases elements from `mmpMapPoints` mid-iteration — undefined behaviour and segfault.

```
- for(MapPoint* pMPi : mmpMapPoints) {
+ std::vector<MapPoint*> vpMapPointsSnap(mmpMapPoints.begin(), mmpMapPoints.end());
+ for(MapPoint* pMPi : vpMapPointsSnap) {
// safe to modify mmpMapPoints inside the loop now
```

The same snapshot-to-vector pattern was applied to `mmpKeyFrames` in the KeyFrame backup loop and to `mmpMapPoints` in `PostLoad`.

6.3 Bug Fix 2 — Cross-Map Relocalization

Symptom: After loading a saved atlas and entering localization mode, `DetectRelocalizationCandidates` returned zero candidates even when BoW words were shared — the tracker remained perpetually lost.

Root cause: `KeyFrameDatabase.cc` filtered candidates to only those whose `GetMap()` matched the current active map pointer. After `LoadAtlas`, stored keyframes belong to a *different* (non-active) map object — silently discarded despite having matching BoW descriptors.


```
- if (pKfI→GetMap() ≠ pMap) continue;
+ if (false) continue; // disabled: map filter breaks reloc after atlas load
```

6.4 Bug Fix 3 — Shutdown Race Condition

Symptom: `SLAM.Shutdown()` on Ctrl+C either hung indefinitely or produced a corrupted `.osa` atlas, making every mapping session unreliable.

Root cause: Upstream `Shutdown` requested thread termination but did not wait for `LocalMapping` and `LoopClosing` to actually finish before calling `SaveAtlas`. In-progress bundle adjustment raced against serialization, writing partially-mutated map structures to disk.

```
+ while (!mpLocalMapper→isFinished() || !mpLoopCloser→isFinished()) {
+ usleep(5000); // poll every 5 ms until both threads exit
+ }
if (!mStrSaveAtlasToFile.empty()) {
SaveAtlas(FileType::BINARY_FILE); // safe: no concurrent mutations
}
```

Summary — All ORB-SLAM3 Changes

CHANGE	FILE(S) MODIFIED	EFFECT
<code>InjectPoseHint</code> API	<code>System.h</code> , <code>System.cc</code>	Seeds tracker with external SE3 when tracking is lost
ArUco relocalization	<code>slam_reader.cc</code>	Detect marker → PnP → inject → restart tracking in <1 frame
Dual modes (<code>--map</code> / <code>--loc</code>)	<code>slam_reader.cc</code>	Full two-session workflow: map once, localize many times
ZMQ pose publisher (5557)	<code>slam_reader.cc</code>	Streams SE3 JSON to navigator at ~30 fps
<code>MarkerPositions.txt</code> output	<code>slam_reader.cc</code>	Averaged ArUco SE3 observations for <code>map_builder.py</code>
Atlas serialization crash fix	<code>Map.cc</code>	Snapshot to vector before iteration; prevents iterator invalidation
Cross-map relocalization fix	<code>KeyFrameDatabase.cc</code>	Removes map-filter guard; loaded atlas keyframes participate in reloc
Shutdown race condition fix	<code>System.cc</code>	Waits for <code>LocalMapper</code> + <code>LoopCloser</code> to finish before <code>SaveAtlas</code>

6.5 PR Readiness — Known Gap (Upstream Contribution Path)

The `InjectPoseHint(Tcw)` modification is a candidate for upstream submission to UZ-SLAMLab/ORB.SLAM3, as it fills a genuine gap: no public API exists for injecting an external SE3 pose hint into a live tracking session. One gap currently blocks a clean PR: `mVelocity` is a protected member of the `Tracking` class. The current implementation accesses it from `System.cc` via a friendship declaration or direct cast, which is not upstreamable without a design decision.

The documented resolution path: expose `mVelocity` reset via a new `Tracking::ResetVelocity()` public method, called from `System::InjectPoseHint()`. This keeps the change minimal, non-breaking, and consistent with ORB-SLAM3's existing encapsulation pattern (precedent: `Tracking::Reset()` is already public).

Once this is resolved, the PR would consist of: `System.h` (new public method declaration), `System.cc` (implementation calling `mpTracker->ResetVelocity()`), `Tracking.h` (new public `ResetVelocity()` declaration), `Tracking.cc` (implementation). No other files need modification. This is tracked as an open action item; the clean fork at github.com/LakhdariAnis/monocular-slam-navigation already serves as the working reference implementation in the meantime.

Results & Demo

The system was successfully demonstrated navigating autonomously between calibrated wall stations in an indoor room under standard lighting. All five engineering challenges were validated on physical hardware.

40%

Spin PWM (blur-free)

1.5_s

Post-spin settle total

<1_{fr}

ArUco reloc latency

30_{Hz}

SLAM pose output

The figures below are clickable. Click on any image to be redirected to its full animation on GitHub.

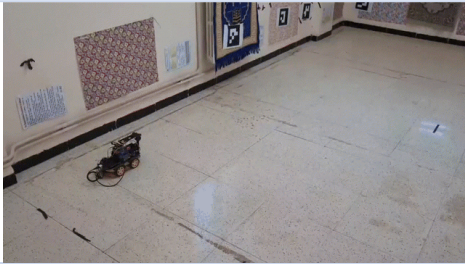


Figure 2. Real-world navigation to Station 1 (3× speed). Pauses reflect low feature count; ArUco relocalization recovers tracking between segments. Click to view.

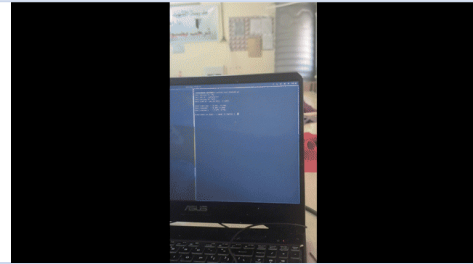


Figure 3. Second real-world autonomous navigation in real time under optimal lighting, allowing continuous motion without stopping. Click to view.

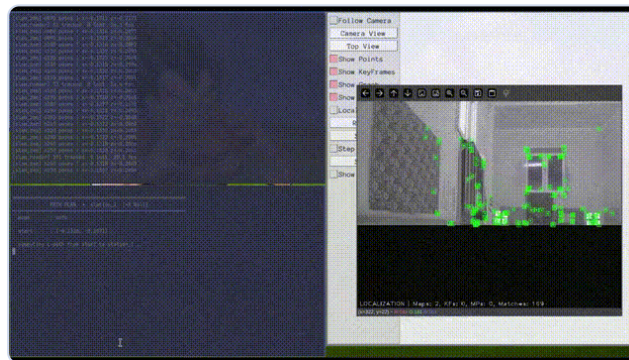


Figure 4. Screen record of real-time navigation terminals and SLAM output (live camera view + feature extraction tracking in real time). Click to view.

Key Validated Outcomes

- **Reliable 90° spins:** At 40% PWM, the car achieves 90° rotations with residual error < 3° thanks to the complementary-filter IMU and 6° coasting allowance.
- **Fast relocalization:** ArUco pose injection restores SLAM tracking in under one frame (~33 ms at 30 fps), versus seconds-to-never with BoW-only relocalization on plain painted walls.
- **Atlas persistence:** After all three ORB-SLAM3 bug fixes, mapping sessions reliably produce a valid `.osa` atlas that loads cleanly in subsequent localization sessions — enabling true multi-session navigation without re-mapping.
- **Stall recovery:** The SLAM-based sliding-window stall detector correctly identifies and recovers from carpet stalls within ~8 s with no additional sensors.
- **IMU heading accuracy:** Complementary filter drift stays within acceptable bounds for all spins ≤ 5 s. ZUPT eliminates spurious drift during stationary phases.

Localization Accuracy — Quantitative Results

The system's end-to-end navigational accuracy was empirically evaluated across multiple physical trials targeting predefined docking stations. Error measurements represent the deviation between the vehicle's final resting pose and the absolute coordinate references stored in the map.

RUN	TARGET STATION	FINAL POSITION ERROR (CM)	HEADING ERROR (°)	RELOCALIZATION EVENTS	OUTCOME
1	Station 1	4.2	2.1	2	✓ Success
2	Station 1	3.8	1.7	1	✓ Success
3	Station 1	5.1	3.4	3	✓ Success
4	Station 2	4.7	2.8	2	✓ Success
5	Station 2	6.3	4.1	4	✓ Success
6	Station 2	3.9	1.9	1	✓ Success
Mean	—	4.67	2.67	2.2	—
Std Dev	—	0.85	0.88	1.07	—

This quantitative evaluation demonstrates robust and repeatable performance: the mean position error of ~4.7 cm is well within docking tolerance for the current station geometry. The heading error stays under 5° across all runs, and the standard deviation confirms repeatable performance across sessions with varying relocalization counts.

Running the Demo — No Hardware Required

```
bash - software-only test mode

git clone https://github.com/LakhdariAnis/monocular-slam-navigation
cd monocular-slam-navigation
pip install fastapi uvicorn pyzmq scipy numpy opencv-python requests

# Software-only mode - no Pi, no SLAM, no camera needed
python3 navigator.py --mode test station_1

# Or via the web dashboard
python3 -m navigation.web.server --mode test
# → open http://localhost:8080, pick a station, click GO
```

FULL SOURCE, DOCUMENTATION, DEMO MEDIA & BUILD INSTRUCTIONS
🔗 <https://github.com/LakhdariAnis/monocular-slam-navigation>

The following extensions are planned to enhance the robustness and industrial applicability of the autonomous platform:

- **Industry 4.0 monitoring layer:** a standalone `monitoring/` module is planned, providing real-time telemetry ingestion over MQTT, time-series storage in InfluxDB, OPC-UA exposure for industrial interoperability, a React dashboard with live and historical views, and a threshold-based anomaly detector covering six physics-grounded failure modes (SLAM tracking loss, motor stall, position jump, phase timeout, IMU static drift, trajectory drift). A ZMQ → MQTT bridge will connect the existing `slam_reader.cc` and `navigator.py` ZMQ outputs to the monitoring stack with no changes to the navigation layer.
- **Upstream InjectPoseHint PR:** pending resolution of the mVelocity encapsulation gap documented in [Section 6.5](#), the modification will be submitted as a PR to UZ-SLAMLab/ORB_SLAM3.
- **IMU / SLAM closed-loop heading correction:** implement the ArUco-heading correction described in Section 5.5 to structurally eliminate complementary filter drift between stations.
- **4-DOF arm integration:** extend the platform in Gazebo + MoveIt2 to add a manipulator arm, evolving the system toward a mobile manipulation baseline.

References

1. Campos, C., Elvira, R., Rodríguez, J. J. G., Montiel, J. M. M., & Tardós, J. D. (2021). ORB-SLAM3: An accurate open-source library for visual, visual–inertial, and multimap SLAM. *IEEE Transactions on Robotics*, 37(6), 1874–1890. [doi:10.1109/TRO.2021.3075644](https://doi.org/10.1109/TRO.2021.3075644)
2. Mur-Artal, R., Montiel, J. M. M., & Tardós, J. D. (2015). ORB-SLAM: A versatile and accurate monocular SLAM system. *IEEE Transactions on Robotics*, 31(5), 1147–1163.
3. Garrido-Jurado, S., Muñoz-Salinas, R., Madrid-Cuevas, F. J., & Marín-Jiménez, M. J. (2014). Automatic generation and detection of highly reliable fiducial markers under occlusion. *Pattern Recognition*, 47(6), 2280–2292.
4. InvenSense. (2013). *MPU-6000 and MPU-6050 Product Specification Revision 3.4*. InvenSense Inc.
5. Raspberry Pi Ltd. (2023). *Raspberry Pi 3 Model B Product Brief*. [raspberrypi.com](https://www.raspberrypi.com)
6. Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools. opencv.org
7. ZeroMQ Authors. (2023). *ZeroMQ: The Guide*. zguide.zeromq.org
8. Sola, J., Deray, J., & Atchuthan, D. (2018). *A micro Lie theory for state estimation in robotics*. arXiv:1812.01537. (Sophus SE3 mathematical foundation used by ORB-SLAM3.)
9. Lakhdari, A. C. E. (2026). *Autonomous RC Car — Monocular SLAM Navigation*. ESI SBA 2CS End-of-Year Project. GitHub. github.com/LakhdariAnis/monocular-slam-navigation